
ENGINEERING CODE SAMPLE / VERSION 1.0

Membership Ops

A readable path from UI action to an authenticated, authorized, transactional renewal request.

NestJS / Prisma / PostgreSQL / Next.js



Contents

A compact navigation map for the implementation.

1. Project purpose
 2. Scope and non-goals
 3. Architecture
 4. Repository structure
 5. Primary request lifecycle
 6. Login and authentication
 7. Authorization
 8. Input validation
 9. Membership business rules
 10. Pricing
 11. Payment boundary
 12. Database design
 13. Prisma access
 14. Transaction semantics
 15. Error handling
 16. Logging and request correlation
 17. Security decisions
 18. Testing strategy
 19. UI architecture
 20. Developer View architecture
 21. Deployment
 22. Known limitations
 23. Production evolution
 24. Review path
- Appendix: exact source map

Membership Ops engineering handbook

Version 1.0 - reviewer documentation

How to use this handbook

Membership Ops is intentionally small enough to hold in your head. This handbook is a map through the implementation, not a replacement for opening the source. Start with the primary request, then follow the code paths named in each chapter.

Design rationale

The primary request is easy to trace through the UI, API, validation, identity, authorization, business rules, provider boundary, database transaction, errors, and logs without requiring a distributed system diagram. The main source files are `apps/web/app/page.tsx`, `apps/api/src/memberships/memberships.controller.ts`, and `apps/api/src/memberships/memberships.service.ts`.

1. Project purpose

Membership Ops is a membership billing application created as an engineering code sample. A signed-in member can inspect membership state, choose a renewal term, select a deterministic demo payment outcome, and see the result reflected in membership and payment history. A reviewer can open Developer View and trace the same behavior to actual source paths.

The application is production-minded in its boundaries rather than production-sized in its scope. It uses real request validation, JWT verification, bcrypt password comparison, typed relational access, integer cents, a database transaction, structured request correlation, and predictable error responses. It does not pretend that a fake payment provider or a single Compose project solves every payment and identity problem.

The primary request is:

```
POST /api/v1/memberships/:membershipId/renew
{
  "months": 1 | 3 | 6 | 12,
  "paymentMethodId": "pm_demo_success" | "pm_demo_declined"
}
```

2. Scope and non-goals

The domain contains users, memberships, and payments. Roles are MEMBER and ADMIN. A member may renew only their own membership. A cancelled membership cannot be renewed. Active memberships extend from their current expiration date; expired memberships extend from the current time.

The sample does not include real payment processing, card data, Redis, Kafka, RabbitMQ, Kubernetes, a microservice split, a plan catalog, complex entitlement states, refresh-token rotation, MFA, password reset, fraud scoring, or a full admin console. These are deliberate boundaries. The point is to show sound decisions on a complete request, not to inflate the architecture.

Production consideration

Add the next capability when a real requirement creates a reason: provider idempotency and reconciliation for payments, a durable identity lifecycle, stronger observability, rate limiting, and integration/browser tests in CI.

3. Architecture

The repository is a pnpm workspace:

```
membership-ops-demo/  
  apps/  
    api/      NestJS, Prisma, PostgreSQL boundary  
    web/      Next.js product UI and Developer View  
  docs/      handbook, decisions, deployment, security  
  scripts/   deterministic source-snippet extraction  
  docker-compose.yml
```

The runtime flow is:

```
Browser  
-> Next.js UI  
-> POST /api/v1/memberships/:membershipId/renew  
-> Nginx or local API port  
-> Nest controller  
-> ValidationPipe and JwtAuthGuard  
-> MembershipsService  
-> Prisma read  
-> pricing function  
-> PaymentGateway  
-> Prisma transaction  
-> normalized HTTP response
```

The API is a modular monolith. auth, memberships, payments, and prisma have clear local responsibilities. The common filter and logging middleware are cross-cutting infrastructure.

There is no application-wide generic framework that hides the actual business path.

4. Repository structure

`apps/api/src/main.ts` creates the Nest app, establishes the `api/v1` prefix, configures CORS, installs the request logger, enables the whitelist validation pipe, and registers the global exception filter. `apps/api/src/app.module.ts` composes feature modules.

`apps/api/src/auth/` contains login, DTO validation, JWT verification, and the `CurrentUser` parameter decorator. `apps/api/src/memberships/` contains the controller, repository, service, renewal DTO, pricing function, and tests. `apps/api/src/payments/` models the provider seam and fake implementation. `apps/api/src/prisma/` owns the Prisma client lifecycle.

`apps/web/app/page.tsx` is intentionally easy to open. It contains the login screen, overview, renewal form, payments view, membership detail view, and Developer View. Its visual system lives in `apps/web/app/globals.css`. `scripts/extract-developer-flow.mjs` reads source markers and writes the generated JSON consumed by the browser.

5. Primary request lifecycle

The request is intentionally linear:

1. The UI selects months and a demo payment ID. 2. `handleRenew` sends the membership ID from the API response and the selected input. 3. Nest maps the URL and body into `MembershipsController.renewMembership`. 4. `ValidationPipe` rejects extra fields and unsupported values. 5. `JwtAuthGuard` verifies the bearer JWT and attaches its subject to the request. 6. `CurrentUser` passes the verified identity to the service. 7. The service loads the membership and checks owner or admin access. 8. The service rejects cancelled state. 9. `calculateRenewalPrice` returns amount and discount in integer cents. 10. `PaymentGateway.charge` runs the fake external boundary. 11. A successful payment is persisted with the membership expiration update in one Prisma transaction. 12. The controller returns a stable success payload.

The request logger records a correlation ID and the filter converts known or unknown failures into a consistent shape.

Relevant source

`apps/api/src/memberships/memberships.controller.ts` first, then jump to `MembershipsService.renewMembership`.

6. Login and authentication

POST /api/v1/auth/login accepts email and password through LoginDto. The global validation pipe checks email shape and a minimum password length. AuthService.login lowercases the email for lookup, retrieves the user through Prisma, and uses bcrypt.compare. A missing user and a wrong password produce the same AUTHENTICATION_FAILED application error.

On success, JwtService.signAsync signs the user ID as sub, along with email and role. The response includes the access token and a safe user object containing ID, email, and role. The password hash is never returned. The browser keeps this short-lived demo token in session storage and sends it as a bearer token.

JwtAuthGuard is the explicit boundary before protected controller methods. It checks the Authorization prefix, verifies signature and expiry using the server-side configured secret, and places the verified identity on request.user. The CurrentUser decorator makes that identity available without allowing a caller to supply it.

Security consideration

Identity is established by the signed token and verified on the server. A body field would let a caller attempt to choose another identity.

7. Authorization

Authentication and authorization are separate. A valid JWT proves who the caller is; it does not prove that the caller may modify an arbitrary membership. In `MembershipsService.renewMembership`, the service loads the membership, returns `MEMBERSHIP_NOT_FOUND` if it does not exist, then compares `membership.userId` with `user.id` for members. Admins are allowed to operate on any membership in this intentionally small role model.

The authorization check is in the service rather than only in the UI or a controller decorator because it protects the business operation regardless of which future entry point calls it. The UI does not hide access rules; it simply presents the authenticated user's own overview.

Potential future policies include organization membership, named permissions, audit records, and a dedicated policy layer. They are not needed for two roles and one resource today.

8. Input validation

RenewMembershipDto uses `@IsIn([1, 3, 6, 12])` for months and `@IsIn(['pm_demo_success', 'pm_demo_declined'])` for the payment scenario. The global `ValidationPipe` enables `whitelist` and `forbidNonWhitelisted`, so unknown body properties are rejected rather than silently accepted.

Validation is deliberately before business logic and before the payment boundary. The fake gateway is not responsible for deciding whether a term is valid. The DTO tests prove both a supported request and rejected values.

In a larger API, validation messages could include a safe field-error array for client forms. Stable top-level error codes should remain consistent even if wording changes.

9. Membership business rules

The service first finds the resource, then authorizes access, then checks membership state. The order gives a clear error for missing records and avoids charging a payment method when the caller is not allowed to act. Cancelled membership produces `MEMBERSHIP_CANCELLED` before pricing or payment. Active and expired membership are both valid renewal inputs.

Renewal starts at:

```
max(current validUntil, current time)
```

That means a member does not lose remaining active time. An expired membership starts from now instead of extending an old date. `addMonths` keeps date arithmetic in one testable function and does not mutate its input date.

The result sets the status to `ACTIVE`, stores the paid term, and returns the updated membership plus payment data. The payment is associated to the membership owner, not a user ID chosen by the browser.

10. Pricing

Pricing is pure domain logic in `apps/api/src/memberships/pricing/pricing.ts`. The discount table is explicit:

Term	Discount	Formula for a \$49.00 monthly plan
1 month	0%	4,900 cents
3 months	5%	13,965 cents
6 months	10%	26,460 cents
12 months	15%	49,980 cents

The function calculates undiscounted cents, rounded discount cents, and final amount cents. It does not use floating-point currency. Keeping this function free from Nest and Prisma makes its four pricing tests fast and direct.

Design rationale

Integer cents make arithmetic deterministic for this domain. A production system may choose a database decimal type for broader monetary requirements, but it must still define rounding and currency rules explicitly.

11. Payment boundary

PaymentGateway is an abstract class token with a charge method. FakePaymentGateway returns a deterministic provider reference for pm_demo_success and throws PaymentDeclinedError for pm_demo_declined. There are no card numbers, secrets, or provider credentials in the project.

The service depends on the abstraction, not the fake class. This makes the external boundary visible and lets tests control success or failure. A real adapter could call a provider SDK, but it would need to map provider response types and errors into domain-level results rather than leak SDK exceptions through the API.

Payment runs after business checks and pricing but before database writes. That keeps a declined demo scenario from creating a payment row or extending membership.

12. Database design

The Prisma schema contains:

- User: unique email, bcrypt hash, role, timestamps, one optional membership, and payments.
- Membership: one-to-one user relation, plan, integer monthly price, status, expiration, timestamps, and payments.
- Payment: user and membership foreign keys, integer amount, term months, status, deterministic provider reference, and creation time.

The database has unique user email and membership userId. Status and expiration have indexes for common operational queries. Payment history has composite indexes by user and membership with creation time. Relations use restrictive deletes for payment history and cascading delete from user to membership as a compact demo policy.

PostgreSQL is appropriate because the domain is relational and the key write must be atomic. Prisma migrations keep schema change explicit and the typed client avoids unsafe string-built SQL.

13. Prisma access

PrismaService extends PrismaClient and connects/disconnects through Nest module lifecycle hooks. MembershipsRepository owns the read shapes used by the feature: lookup by ID and current-user overview with recent payments. The service does not know how Prisma filters or includes are written for these reads.

The transaction writes use the transaction client passed to the callback. That detail matters: using the root Prisma client for a write inside the callback would leave the transaction boundary. The code calls tx.payment.create and tx.membership.update on the same transaction client.

14. Transaction semantics

After fake payment success, `this.prisma.$transaction` creates the Payment row and updates `Membership.validUntil`. If the second write fails, Prisma rolls back the first write. The service test mocks the transaction callback to prove both operations are requested together and separately verifies that a transaction failure does not return a success result.

This transaction does not make a remote payment atomic with PostgreSQL. The external provider has already seen the charge before the database callback begins. That is not a defect hidden by the sample; it is the exact discussion point the architecture exposes.

Production consideration

Generate an idempotency key per renewal intent, persist a pending payment intent, make provider calls retry-safe, accept signed webhooks, reconcile asynchronous state, use an outbox for downstream work, and define compensating behavior for a successful charge whose local write cannot commit.

15. Error handling

Known failures use `ApplicationError` and stable codes:

Code	Status	Example
VALIDATION_FAILED	400	unsupported body value
AUTHENTICATION_FAILED	401	missing/invalid login or JWT
MEMBERSHIP_NOT_FOUND	404	unknown membership
MEMBERSHIP_ACCESS_DENIED	403	member targets another member
MEMBERSHIP_CANCELLED	409	cancelled state
PAYMENT_DECLINED	402	demo decline
INTERNAL_ERROR	500	unexpected failure

`ApplicationExceptionHandler` recognizes application errors and selected Nest HTTP exceptions. It returns `statusCode`, `code`, `message`, and `requestId`. Unexpected errors are logged internally but expose only a generic client message. Stack traces and Prisma internals are not serialized into the response.

16. Logging and request correlation

requestLogger reads a bounded incoming X-Request-Id or generates a UUID. It returns the same ID in the response header and logs method, route, status, duration, and authenticated user ID when available. The exception filter adds stable code and internal diagnostic text for server-side diagnosis.

The implementation intentionally avoids request-body logging and header serialization. It never intentionally logs passwords, password hashes, JWTs, authorization headers, database URLs, or credentials. In production, the same fields should flow into a redacting JSON logger, centralized storage, and a retention policy.

17. Security decisions

The main security boundary is server-side identity and authorization. Environment variables hold database configuration and signing material. `.env` is ignored, and the example file contains placeholders only. Bcrypt protects stored passwords. Validation rejects extra fields and unsupported values. Prisma operations are typed and parameterized.

The browser uses a short-lived bearer token in session storage for a transparent demo flow. A real product could use a secure `HttpOnly SameSite` cookie and CSRF protection, a BFF, or an OIDC client depending on deployment. The handbook records the actual choice and its limitation instead of implying that a demo token model is a complete identity platform.

Next emits basic security headers. The production proxy disables caching for authenticated API paths and preserves `X-Request-Id`. The Euronodes origin certificate covers `vgym.mcorucu.com`, and Cloudflare Proxy is enabled with Full (strict) TLS. Application login remains the product access boundary.

18. Testing strategy

Tests focus on behavior and decision points:

- pricing for 1, 3, 6, and 12 months
- date extension without mutation
- DTO rejection for invalid terms and payment methods
- generic invalid password behavior
- unauthenticated guard rejection and verified identity attachment
- member ownership denial
- cancelled membership rejection
- payment decline with no database transaction
- successful payment and membership update in one transaction
- transaction failure propagation

The suite is unit-oriented and fast. It does not claim arbitrary 100% coverage. A production pipeline should add a PostgreSQL-backed integration test covering login, migrations, renewal persistence, and rollback, plus a browser smoke test for login and both payment scenarios.

19. UI architecture

The UI is a real Next.js client page with a login view and an authenticated operational workspace. It calls the API for login, overview, renewal, and refreshed state. It does not invent a membership ID or user ID. The membership ID used by renewal comes from the API overview response.

Overview includes current user, plan, status, valid-until date, monthly price, recent payment, a clear primary renewal CTA, term selection, discount preview, and demo payment scenarios. Membership and Payments views provide a compact read model. The UI shows stable error codes and messages returned by the API.

The CSS adapts the warm workspace and dark shell language of the Command Center reference while keeping the content and component system independent. Focus styles, labels, fieldsets, responsive grids, and text-based status all support accessibility.

20. Developer View architecture

Developer View presents the request as 11 stages: request, validation, authentication, authorization, business logic, pricing, payment, database transaction, response, errors, and logging. Each stage shows a source path and a snippet.

The snippets are not manually copied into the page. `scripts/extract-developer-flow.mjs` reads source files from the repository using stable start/end markers and writes `apps/web/public/developer-flow.json`. The root `dev:web` and `build` scripts run extraction before `Next` starts or compiles. This is a small deterministic mechanism with an honest limitation: if a source shape changes without updating a marker, the extractor must be fixed.

Developer View helps a reviewer understand the code but explicitly does not replace opening the real source.

21. Deployment

Local Compose starts PostgreSQL on localhost port 5433. Production uses an isolated Compose project named `vgym_membership_ops`, with a dedicated database container, application environment, generated credentials, named volume, and project-specific internal network. Its source directory is `/srv/vgym-membership-ops`; API and web join the Euronodes server's existing `eskisehiraraba_prod_edge` network only for proxy routing.

The verified Euronodes host has a shared Nginx proxy on ports 80 and 443. The safe integration is an additional server block, not replacement of the global proxy. `/api/v1/` routes to the API container and `/` routes to the web container. The proxy is tested and reloaded in place after the new configuration and certificate are ready. Unrelated Compose projects and resources are not part of the release command.

22. Known limitations

The fake payment boundary, simple JWT lifecycle, minimal logs, small membership state machine, lack of fraud controls, lack of rate limiting, and demo seed are intentional. See [docs/KNOWN_LIMITATIONS.md](#) for the complete list.

The most important limitation is distributed payment consistency. PostgreSQL can guarantee the local payment and membership rows commit together. It cannot undo or include a remote provider charge automatically. Production correctness comes from idempotency, durable intent state, provider webhooks, retries, reconciliation, and compensating decisions.

23. Production evolution

A sensible next sequence is:

1. Add request idempotency and a payment intent state machine. 2. Add provider webhook verification and reconciliation jobs. 3. Add integration tests against PostgreSQL and browser smoke tests in CI. 4. Add structured redacting logs, metrics, tracing, and alerting. 5. Replace demo identity lifecycle with OIDC or secure cookie sessions, refresh rotation, MFA, and recovery. 6. Add rate limiting and account protection based on observed abuse risk. 7. Introduce an outbox only when downstream asynchronous work exists.

This order preserves the understandable monolith while addressing the highest correctness and operational risks first.

24. Review path

Start in the UI at Overview and trigger a successful renewal. Inspect the browser request, then follow the controller, DTO, global validation pipe, JWT guard, CurrentUser, service authorization check, pricing function, and gateway abstraction. The source map in the appendix provides the corresponding paths.

Next, inspect the transaction callback and compare local rollback behavior with the external payment boundary. The declined scenario demonstrates that membership expiration remains unchanged. The behavior tests, deployment shape, and known limitations document the remaining operational boundaries.

Production consideration

It does not claim that an external payment provider participates in a PostgreSQL transaction. That boundary requires a production payment state machine and reconciliation design.

Appendix: exact source map

Concern	File	Function/class
UI trigger	apps/web/app/page.tsx	handleRenew, OverviewView
Request	apps/api/src/memberships/memberships.controller.ts	MembershipsController.renewMembership
Validation	apps/api/src/memberships/dto/renew-membership.dto.ts	RenewMembershipDto
Authentication	apps/api/src/auth/guards/jwt-auth.guard.ts	JwtAuthGuard.canActivate
Login and password comparison	apps/api/src/auth/auth.service.ts	AuthService.login
Identity	apps/api/src/auth/decorators/current-user.decorator.ts	CurrentUser
Authorization	apps/api/src/memberships/memberships.service.ts	member ownership check
Business logic	apps/api/src/memberships/memberships.service.ts	renewMembership
Pricing	apps/api/src/memberships/pricing/pricing.ts	calculateRenewalPrice
Payment	apps/api/src/payments/fake-payment.gateway.ts	FakePaymentGateway.charge
Database	apps/api/src/memberships/memberships.service.ts	this.prisma.\$transaction
Response	apps/api/src/memberships/memberships.controller.ts	controller return
Errors	apps/api/src/common/filters/application-exception.filter.ts	catch
Logging	apps/api/src/common/logging/request-logger.middleware.ts	requestLogger
Important tests	apps/api/src/memberships/memberships.service.spec.ts	renewal behavior suite